θ–Mellin 表示的反射对称

Lean 4 证明解读与使用说明：`ThetaMellinSymmetry.lean`

本例取自利用 Jacobi θ 模反演推导 ζ 函数方程的思路，但目前只完成其中最后的代数对称步骤。本文先说明它在完整推导中的位置，再逐条解释实际代码和 VS Code 的操作方法。程序没有把尚未完成的积分、换序或解析延拓作为未经证明的公理。因此，本例可以作为理解分析证明接口的材料，不能称为 ζ 函数方程的完整形式化证明。

1 数学背景与实际目标

1.1 完整推导为何会出现这种对称形式

取实参数级数

$$\theta(t)=\sum_{n\in\mathbb{Z}}e^{-\pi n^2t},\qquad \omega(t)=\frac{\theta(t)-1}{2}=\sum_{n\geq 1}e^{-\pi n^2t},\quad t>0.$$

通常数学推导先在 $\operatorname{Re}s>1$ 建立 Mellin 表示

$$\pi^{-s/2}\Gamma(s/2)\zeta(s)=\int_0^\infty \omega(t)t^{s/2-1}\,dt.$$

这里的 t^z 对 $t>0$ 按 $\exp(z\log t)$ 理解。用模反演 $\theta(t)=t^{-1/2}\theta(1/t)$，得到

$$\omega(t)=\frac{t^{-1/2}-1}{2}+t^{-1/2}\omega(1/t).$$

在 $t=1$ 拆分积分，并在小区间代换 $u=1/t$，有理修正项为

$$\frac{1}{2}\int_0^1\left(t^{s/2-3/2}-t^{s/2-1}\right)dt=\frac{1}{s-1}-\frac{1}{s}.$$

令

$$A(s)=\int_1^\infty \omega(t)t^{s/2-1}\,dt,$$

即可得到形如

$$\pi^{-s/2}\Gamma(s/2)\zeta(s)=-\frac{1}{s}-\frac{1}{1-s}+A(s)+A(1-s) \tag{1}$$

的表示。上述积分计算首先在原有收敛半平面进行；继续到更大区域，还需证明 A 的解析性及新表示与原函数的一致性。**本节只是通常数学推导的背景说明，不是本程序已经完成的形式化步骤**。

1.2 程序真正证明的是一个代数模型

对任意复函数 $A:\mathbb{C}\to\mathbb{C}$，定义

$$F_A(s)=-\frac{1}{s}-\frac{1}{1-s}+A(s)+A(1-s). \tag{2}$$

程序证明

$$F_A(1-s)=F_A(s). \tag{3}$$

这个结论不要求 A 连续、可微或解析，原因是反射只交换两项分式与两项函数值。通常先对 $s\neq 0,1$ 写下

$$\begin{aligned}F_A(1-s)&=-\frac{1}{1-s}-\frac{1}{1-(1-s)}+A(1-s)+A(1-(1-s))\\&=-\frac{1}{1-s}-\frac{1}{s}+A(1-s)+A(s)\\&=F_A(s).\end{aligned}$$

Lean 中复数除法在零分母处也有约定值，所以程序的代数等式对每个复数 s 成立。这不意味着式 (1) 左侧的亚纯函数在 $0,1$ 处没有极点。

2 完整源码与准备部分

```
import Mathlib.Data.Complex.Basic
import Mathlib.Tactic

/-!
# θ–Mellin 表示的反射对称：最后的代数步骤

重要：本文件不是 ζ 函数方程的完整形式化证明。
这里的 A 是任意复函数，尚未证明它等于 θ–Mellin 推导中的含参量积分，
也没有证明下面的模型等于 π^(-s/2)Γ(s/2)ζ(s)。
所以本文件只核验推导最后的对称性代数，不把分析接口藏进假设或公理。
核验范围与尚待完成的分析接口见同目录 README.md。
-/

namespace Progressive

-- 定义对称积分表示的代数模型；A 将来应由 [1,∞) 上的积分定义。
noncomputable def thetaMellinModel (A : ℂ → ℂ) (s : ℂ) : ℂ :=
  -1 / s - 1 / (1 - s) + A s + A (1 - s)

-- 核验最后一步：替换 s 为 1-s 只会交换两项分式和两项 A。
theorem thetaMellinModel_symmetric (A : ℂ → ℂ) (s : ℂ) :
    thetaMellinModel A (1 - s) = thetaMellinModel A s := by
  -- 先明确写出反射两次回到原点 s，而不是调用 ζ 的函数方程。
  have hReflection : (1 : ℂ) - (1 - s) = s := by ring
  -- 展开模型，显示四个需要比较的项。
  unfold thetaMellinModel
  -- 化简反射两次后的自变量。
  rw [hReflection]
  -- 剩下只是同一批加数的排列，按交换律和结合律核验。
  ring

end Progressive
```

左侧行号只用于阅读，不属于程序。代码直接从同目录的实际文件读取，避免另存一份证明后不同步。

2.1 导入复数和证明策略

```
import Mathlib.Data.Complex.Basic
import Mathlib.Tactic
```

第一行提供复数类型 `ℂ` 及其代数结构；第二行提供包括 `ring` 在内的证明策略。本例并未导入并调用一个现成的 ζ 函数方程来结束证明。所需的数学基础只是复数的加法、减法、除法及代数恒等变形。

2.2 注释与名字空间

`/-! ... -/` 是模块说明，`--` 后是单行注释，两者供人阅读，不参与证明。`namespace Progressive` 为定义和定理添加名称前缀；最后的 `end Progressive` 结束这一范围。例如定理的完整名称为 `Progressive.thetaMellinModel_symmetric`。名字空间不必与目录或文件名称相同，因此按内容更改文件名后仍可保留这个前缀。

3 逐项解释模型定义

```
noncomputable def thetaMellinModel (A : ℂ → ℂ) (s : ℂ) : ℂ :=
  -1 / s - 1 / (1 - s) + A s + A (1 - s)
```

`def` 与定义名称

`def` 引入定义，不是声明一个待证明的结论。`thetaMellinModel` 是式 (2) 的名称。

`noncomputable`

表示这里不要求生成可执行的数值计算程序。Mathlib 的实数、复数理论包含非计算性构造；这个标记不等于证明不严格，也不允许跳过逻辑检查。

`(A : ℂ → ℂ)`

声明 A 是任意从复数到复数的函数，不隐含它来自积分，也不隐含解析性。

`(s : ℂ) : ℂ`

前半段声明复数参数 s，后半段说明定义的输出也是复数。

`A s` 与 `A (1 - s)`

是函数应用，分别对应 $A(s)$ 与 $A(1-s)$。空格不是乘法；乘法需写 `*`。

`:=`

给出定义的具体表达式。数字 1 按上下文解释为复数中的单位元。

3.1 零分母的约定与解析极点的区别

Lean 的域运算是全定义的，约定 $0^{-1} = 0$，因而 $a/0 = 0$。模型在 $s = 0, 1$ 处仍可按这一约定取值；程序对这些点也可证明代数对称。然而亚纯函数的极点描述的是邻域内的解析行为，不能被某个点的补值消除。若要把模型与完备化 ζ 联系起来，应在非极点处识别函数值，或在相应的亚纯框架中处理极点。本例没有证明这种识别，因此不能从它推出有关 ζ 的极点结论。

4 逐条解释反射对称的证明

4.1 读出待证命题

```
theorem thetaMellinModel_symmetric (A : ℂ → ℂ) (s : ℂ) :
    thetaMellinModel A (1 - s) = thetaMellinModel A s := by
```

`theorem` 声明定理，参数之后的冒号引出目标。`:= by` 表示用策略式证明完成该目标。因为 A, s 都没有额外限制，这个定理对任意复函数 A 与任意复数 s 成立。开始时的目标可概括为：

```
A : ℂ → ℂ
s : ℂ
⊢ thetaMellinModel A (1 - s) = thetaMellinModel A s
```

⊢ 上方是参数与已有事实，下方是待证目标。

4.2 建立反射两次回到原变量的局部等式

```
have hReflection : (1 : ℂ) - (1 - s) = s := by ring
```

`have` 引入有证明的局部事实；`hReflection` 是这条事实的名称，不是另一个复数变量。`(1 : ℂ)` 明确数字 1 的类型。目标 $1 - (1 - s) = s$ 由 `ring` 进行多项式恒等整理后证明。这里调用的是复数环的代数规则，不是任何特殊函数的性质。

建立后，上下文中可用

```
hReflection : (1 : ℂ) - (1 - s) = s
```

而外层主目标暂时仍是模型相等。局部等式证明结束，不等于外层定理已经结束。

4.3 展开模型定义

```
unfold thetaMellinModel
```

`unfold` 将两侧模型名称替换为定义表达式。目标对应于

$$-\frac{1}{1-s} - \frac{1}{1-(1-s)} + A(1-s) + A(1-(1-s)) = -\frac{1}{s} - \frac{1}{1-s} + A(s) + A(1-s).$$

此时必须把 $A(1-(1-s))$ 与 $A(s)$ 对齐，不能因为 A 未指定就任意改换它的自变量。

4.4 按已证等式改写自变量

```
rw [hReflection]
```

`rw` 按等式从左向右改写，把目标中出现的 $1-(1-s)$ 换成 s。这同时处理分母和函数自变量中的对应表达式，使用了相等对象可在表达式中替换的规则。改写后的数学目标是

$$-\frac{1}{1-s}-\frac{1}{s}+A(1-s)+A(s)=-\frac{1}{s}-\frac{1}{1-s}+A(s)+A(1-s).$$

这一步不需要 A 连续或可微；对于任意函数，相等的输入必有相等的输出。

4.5 整理同一组加数

```
ring
```

两边现在只是同一组加数的重新排列。`ring` 用环的恒等运算完成整理，将函数值及倒数表达式视为代数子表达式。它没有在这里通分消去分母，也没有假设分母非零，所以不需要 `s ≠ 0` 或 `s ≠ 1`。它更没有证明积分收敛或解析延拓。策略产生的证明仍交由 Lean 内核检查。此行之后，当前定理不再有待证目标。

5 为什么这还不是完整的函数方程证明

要把式 (3) 用于完备化 ζ，需要先证明式 (1) 的分析来源。至少还需完成以下内容：

1. 将 Jacobi θ 模反演与所用实参数级数定义对应起来，核对归一化常数。
2. 从 Γ 积分和 ζ 级数建立 Mellin 表示，证明求和、积分交换所需的绝对可积性。
3. 严格完成积分分拆、变量代换和有理修正项的计算。
4. 用指数衰减证明 $A(s)$ 在参数紧集上满足共同可积控制，进而建立解析性。
5. 在公共区域把新表示与库中的 Γ,ζ 定义识别，再由解析延拓唯一性得到相应函数方程。

这些项目是待实现的数学接口清单，不是本定理偷偷加入的前提。程序中的 A 始终只是任意复函数，没有假定它满足第 1 至第 5 项。因此，当前代码的完整证明范围虽小，却是清楚而自洽的。若希望第三个例程最终成为完整的分析证明，需要逐项补齐上述内容，不能仅改名或扩大说明文字。

6 VS Code 操作与截图建议

6.1 打开项目并选择文件

在 VS Code 中打开含 `lakefile.toml` 和 `lean-toolchain` 的项目根目录，即本仓库中的 “形式化证明-使用 Lean4 核验本书数学推导”，不要只打开本子目录。进入 “Lean 入门示例”，打开 `ThetaMellinSymmetry.lean`。项目使用 Lean 4，依赖版本由配置固定。

打开命令面板（macOS 为 `Cmd+Shift+P`），搜索 `InfoView`，选择 Lean 4 的显示或切换 InfoView 命令。保存文件，等待检查结束。

6.2 依次观察四个位置

1. `have hReflection ...` 之后：观察新增的局部等式与仍需完成的主目标。
2. `unfold thetaMellinModel` 之后：观察模型展开为四项表达式。
3. `rw [hReflection]` 之后：观察两侧函数自变量已对齐，仅余加数排列。
4. 最后一行 `ring` 之后：观察 `Tactic state` 是否显示 `No goals`。

可截取编辑器中的证明代码与旁边 InfoView 同屏画面，保留文件名和光标位置，这样读者能够把所见状态与具体指令对应起来。本说明暂不放置未经实际操作取得的截图；后续可根据作者审核时提供的操作截图补充。

6.3 解释证明通过的显示

`No goals` 表示当前位置没有未完成的策略目标；`Goals accomplished!` 若出现，应结合它所属的定理及位置理解。`Expected type` 显示的是表达式预期具有的类型，不应把其中显示的 $\vdash$ 一律当作未完成目标。在局部 `have` 的内部目标结束时，外层主目标可能尚未结束。最终还需等待整个文件检查完成，确认没有错误信息。

`No info found` 通常只是当前位置没有可显示的信息。可将光标移到最后一行证明指令后，并确认文件已保存、Lean 4 扩展已加载项目。这条信息本身不表示证明失败，也不表示证明通过。

7 终端核验和公理依赖

7.1 核验这个文件

在项目根目录执行：

```
lake env lean Lean 入门示例/ThetaMellinSymmetry.lean
```

命令成功结束且没有错误，表示本文件通过检查。本例没有专门的打印命令，成功时通常没有额外输出；应等待终端提示符重新出现，不能仅凭暂时无输出判断完成。若只出现依赖仓库 `has local changes` 警告，它不是证明错误，但说明本地依赖有改动，应与数学证明是否通过区分。

核验默认入口及其导入模块可运行：

```
lake build
```

统一构建通过不表示目录中每个未导入的私人练习文件也被检查。

7.2 检查标准公理依赖

先运行 `lake build`，再在项目内另建临时检查文件：

```
import «Lean 入门示例 ».ThetaMellinSymmetry

#print axioms Progressive.thetaMellinModel_symmetric
```

用 `lake env lean` 检查该临时文件，报告的是这个定理及其依赖所用的公理。`propext`、`Classical.choice`、`Quot.sound` 是 Mathlib 中常见的形式系统基础；它们不是要求读者补齐的证明缺口。若出现 `sorryAx`，则需追查是否有未完成证明。本例没有使用 `sorry`，也没有把函数方程或积分识别作为额外公理。

8 与前两个例程的配合

3–4–1 例程展示“已有三角恒等式、代数整理、非负性”的完整小证明；von Mangoldt 例程进一步展示归纳、有限和、卷积与反演。本例虽然处在更深的分析问题背景中，实际代码只包含一个短的代数证明，因此不能按背景难度把它称为已完成的综合分析证明。

它当前的学习价值在于：明确地区分“构造了具有对称性的模型”和“证明原函数等于这个模型”。只有后一个识别也被严格完成，模型的对称性才能成为目标函数的函数方程。这一点既适用于阅读书中推导，也适用于审阅计算机或 AI 给出的证明。